
ICFOSS -Internship 2026

Assignment 1

Read the instructions carefully.

If you are interested in doing internship at ICFOSS a sound knowledge of Linux operating system is required. This assignment is to teach you Linux command line tools and find out your aptitude for experimentation on FOSS platforms.

If you have not worked with a FOSS platform before, you can install a Linux os such as Ubuntu 24.04 on your PC and experiment. Or you can install it on a virtual machine such as virtual-box. Look at this tutorial (<https://www.geeksforgeeks.org/how-to-install-ubuntu-on-virtualbox/>)

Instructions for the assignment.

Install a Linux distribution on your PC either as a native install or virtual install.

Complete the following exercise on you PC/virtual machine. Then upload a file containing your experiments. Detailed instructions on how to capture and upload the history of your experiments is given on the last page of this document. (If you are short listed, you can expect interview questions from your experiments with Linux)

Read the material carefully. Do every command listed in a terminal below. If your get an error try to fix it by referring to internet. **Attempt one section per day.** (This is important, There is a learning curve for command line tools. So take time and finish it over a week. Your time stamp will be uploaded)

Uploading instructions at the end of the file No other queries regarding this will be entertained.

The Linux operating system

The major components of the Linux operating system are show in figure 1. Apart from these there are file systems networking stack etc attached to the system.

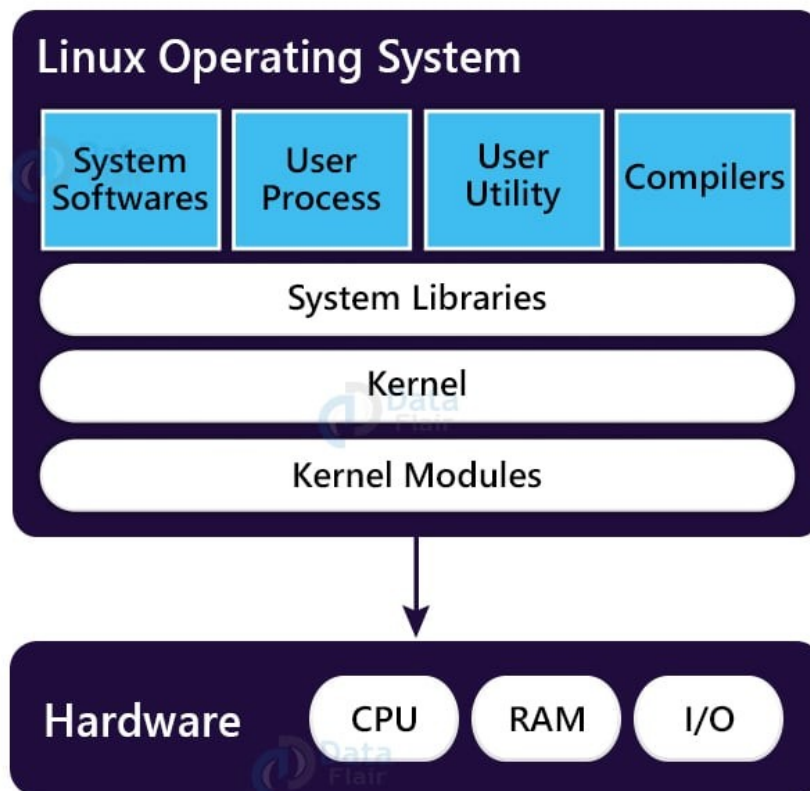


Figure 1

The kernel

The kernel of Linux is the heart of the operating system: it allocates time and memory to programs and handles the filestore and communications in response to system calls.

The shell

The shell acts as an interface between the user and the kernel. When a user logs in, the login program checks the username and password, and then starts another program called the shell. The shell is a command line interpreter (CLI). It interprets the commands the user types in and arranges for them to be carried out. The commands are themselves programs: when they terminate, the shell gives the user another prompt (\$ on our systems).

The adept user can customize his/her own shell, and users can use different shells on the same machine. Some of the available shells are bash, csh etc

Filename Completion - By typing part of the name of a command, filename or directory and pressing the [Tab] key, the bash shell will complete the rest of the name automatically. If the shell finds more than one name beginning with those letters you have typed, it will beep, prompting you to type a few more letters before pressing the tab key again.

History - The shell keeps a list of the commands you have typed in. If

you need to repeat a command, use the cursor keys to scroll up and down the list or type history for a list of previous commands.
If you are new to the shell please see this tutorial <https://www.w3schools.com/bash/>

Files and processes

Everything in Linux is either a file or a process.

A process is an executing program identified by a unique PID (process identifier).

A file is a collection of data. They are created by users using text editors, running compilers etc.

Examples of files:

- a document (report, essay etc.)
- the text of a program written in some high-level programming language
- instructions comprehensible directly to the machine and incomprehensible to a casual user, for example, a collection of binary digits (an executable or binary file);
- a directory, containing information about its contents, which may be a mixture of other directories (subdirectories) and ordinary files.

The Directory Structure

All the files are grouped together in the directory structure. The file-system is arranged in a hierarchical structure, like an inverted tree. The top of the hierarchy is traditionally called **root**.

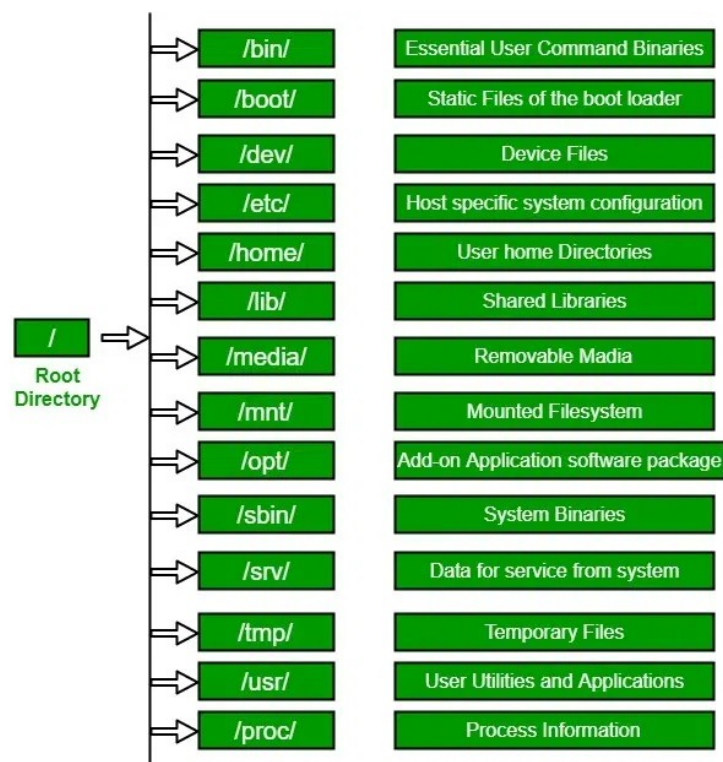


Figure 3

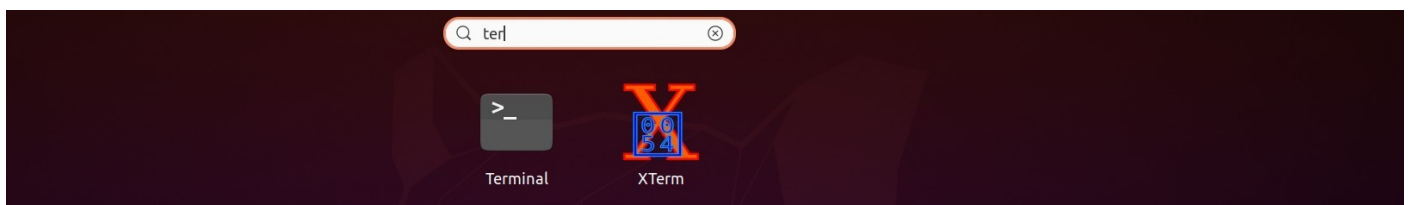
Read the following tutorial if you are new to Linux.

<https://www.geeksforgeeks.org/linux-unix/linux-file-hierarchy-structure/>

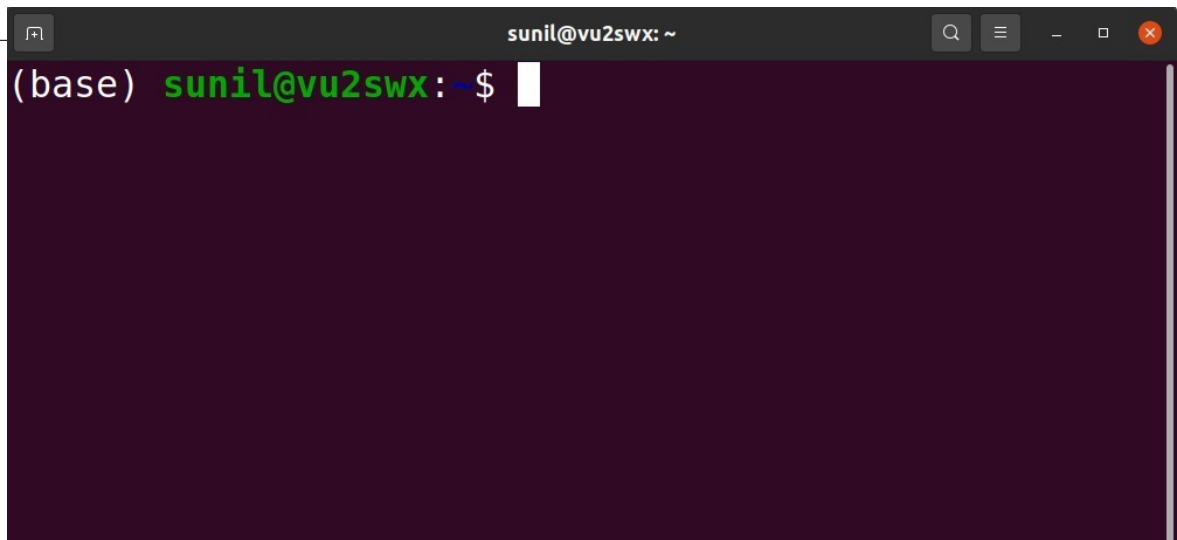
A clear understanding of this will help you later on.

Starting an terminal session

To start an terminal session, click on the Terminal icon on your desktop, or from the drop-down menus



An terminal window will appear with a Linux prompt, waiting for you to start entering commands.

A terminal window with a dark purple background. The title bar at the top shows 'sunil@vu2swx: ~' and standard window controls. The prompt is '(base) sunil@vu2swx: ~\$' with a white cursor.

The assignment begins here. You have to try out each of the commands given below and the history of your experiments has to be uploaded.

1.1 Listing files and directories

Read this if you want to learn more about command prompt
<https://linuxsimply.com/command-prompt-in-linux/>

ls (list)

When you first login, your current working directory is your home directory. Your home directory has the same name as your user-name, It is where your personal files and subdirectories are saved. ([Make sure that the username displayed is your name](#))

To find out what is in your home directory, type

```
$ ls
```

The **ls** command lists the contents of your current working directory.

There may be no files visible in your home directory, in which case, the Linux prompt will be returned. Alternatively, there may already be some files inserted by the System Administrator when your account was created.

ls does not, in fact, cause all the files in your home directory to be listed, but only those ones whose name does not begin with a dot (.) Files beginning with a dot (.) are known as hidden files and usually contain important program configuration information. They are hidden because you should not change them unless you are very familiar with Linux!!!

To list all files in your home directory including those whose names begin with a dot, type

```
$ ls -a
```

ls is an example of a command which can take options: **-a** is an example of an option. The options change the behavior of the command. There are online manual pages that tell you which options a particular command can take, and how each option modifies the

behavior of the command. (See later in this tutorial)

1.2 Making Directories

mkdir (make directory)

We will now make a subdirectory in your home directory to hold the files you will be creating and using in the course of this tutorial. To make a subdirectory called **icfoss** in your current working directory type

```
$ mkdir icfoss
```

To see the directory you have just created, type

```
$ ls
```

1.3 Changing to a different directory

cd (change directory)

The command **cd** *directory* means change the current working directory to 'directory'. The current working directory may be thought of as the directory you are in, i.e. your current position in the file-system tree.

To change to the directory you have just made, type

```
$ cd icfoss
```

Type **ls** to see the contents (which should be empty)

You can just type **cd** to return to your home folder any time.

Exercise 1a

Make another directory inside the **icfoss** directory called **backups**. move inside and then go back to home folder

1.4 The directories . and ..

Still in the **icfoss** directory, type

```
$ ls -a
```

As you can see, in the **icfoss** directory (and in all other directories), there are two special directories called **(.)** and **(..)**

In Linux, **(.)** means the current directory, so typing

```
$ cd .
```

NOTE: there is a space between cd and the dot

means stay where you are (the **icfoss** directory).

This may not seem very useful at first, but using (.) as the name of the current directory will save a lot of typing, as we shall see later in the tutorial.

(..) means the parent of the current directory, so typing

```
$ cd ..
```

will take you one directory up the hierarchy (back to your home directory). Try it now.

Note: typing **cd** with no argument always returns you to your home directory. This is very useful if you are lost in the file system.

1.5 Pathnames

pwd (print working directory)

Path names enable you to work out where you are in relation to the whole file-system. For example, to find out the absolute pathname of your home-directory, type **cd** to get back to your home-directory and then type

```
$ pwd
```

The full pathname will look something like this -

```
/home/unityid
```

which means that unityid (your home directory) is in the directory /home.

Exercise 1b

Use the commands **ls**, **pwd** and **cd** to explore the file system. Try the following and observe the contents of each folder.

```
$ cd /root
```

```
$ cd /etc
```

```
$ cd /home
```

(Remember, if you get lost, type **cd** by itself to return to your home-directory)

1.6 More about home directories and pathnames

Understanding pathnames

First type **cd** to get back to your home-directory, then type

```
$ ls icfoss
```

to list the contents of your icfoss directory. Now type

```
$ ls backups
```

You will get a message like this -

```
backups: No such file or directory
```

The reason is, **backups** is not in your current working directory. To use a command on a file (or directory) not in the current working directory (the directory you are currently in), you must either **cd** to the correct directory, or specify its full pathname. To list the contents of your backups directory, you must type

```
$ ls icfoss/backups
```

~ (your home directory)

Home directories can also be referred to by the tilde ~ character. It can be used to specify paths starting at your home directory. So typing

```
$ ls ~/icfoss
```

will list the contents of your icfoss directory, no matter where you currently are in the file system.

What do you think

```
$ ls ~
```

would list?

What do you think

```
$ ls ~/..
```

would list?

Summary

command	meaning
ls	list files and directories
ls -a	list all files and directories
mkdir	make a directory
cd directory	change to named directory
cd	change to home-directory
cd ~	change to home-directory
cd ..	change to parent directory
pwd	display the path of the current directory

2.1 Copying Files

cp (copy)

cp *file1 file2* is the command which makes a copy of **file1** in the current working directory and calls it **file2**

What we are going to do now, is to take a file stored in an open access area of the file system(find out what is this ?), and use the **cp** command to copy it to your icfoss directory.

First, **cd** to your icfoss directory.

```
$ cd ~/icfoss
```

Then at the Linux prompt, type,

```
$ cp /var/log/syslog syslog.txt
```

The above command means copy the file **syslog** to the current directory

Exercise 2a

Create a backup of your **syslog.txt** file by copying it to a file called **syslog.bak**

2.2 Moving files

mv (move)

mv *file1 file2* moves (or renames) **file1** to **file2**

To move a file from one place to another, use the **mv** command. This has the effect of moving rather than copying the file, so you end up with only one file rather than two.

It can also be used to rename a file, by moving the file to the same directory, but giving it a different name.

We are now going to move the file **syslog.bak** to your backup directory.

First, change directories to your icfoss directory (can you remember how?). Then, inside the **icfoss** directory, type

```
$ mv syslog.bak backups
```

Type **ls** and **ls backups** to see if it has worked.

2.3 Removing files and directories

rm (remove), rmdir (remove directory)

To delete (remove) a file, use the **rm** command. As an example, we are going to create a copy of the **syslog.txt** file then delete it.

Inside your **icfoss** directory, type

```
$ cp syslog.txt tempfile.txt
$ ls #to check if it has created the file
$ rm tempfile.txt
$ ls #to check if it has deleted the file
```

Note: The matter typed after **#** is not interpreted by shell.

You can use the **rmdir** command to remove a directory (make sure it is empty first). Try to remove the **backups** directory. You will not be able to do so ,since Linux will not let you remove a non-empty directory.

Exercise 2b

Create a directory called **tempstuff** using **mkdir** , then remove it using the **rmdir** command.

2.4 Displaying the contents of a file on the screen

clear (clear screen)

Before you start the next section, you may like to clear the terminal window of the previous commands so the output of the following commands can be clearly understood.

At the prompt, type

```
$ clear
```

This will clear all text and leave you with the **\$** prompt at the top of the window.

cat (concatenate)

The command **cat** can be used to display the contents of a file on the screen. Make sure that the file `syslog.txt` is in your home folder .Type

```
$ cat syslog.txt
```

As you can see, the file is longer than the size of the window, so it scrolls past making it unreadable.

less

The command **less** writes the contents of a file onto the screen a page at a time. Type

```
$ less syslog.txt
```

Press the **[space-bar]** if you want to see another page, type **[q]** if you want to quit reading. As you can see, **less** is used in preference to **cat** for long files.

head

The **head** command writes the first ten lines of a file to the screen.

First clear the screen then type

```
$ head syslog.txt
```

Then type

```
$ head -5 syslog.txt
```

What difference did the -5 do to the head command?

tail

The **tail** command writes the last ten lines of a file to the screen.

Clear the screen and type

```
$ tail syslog.txt
```

How can you view the last 15 lines of the file? Find out from internet.

2.5 Searching the contents of a file

Simple searching using less

Using **less**, you can search through a text file for a keyword (pattern). For example, to search through `syslog.txt` for the word 'syslog', type

```
$ less syslog.txt
```

then, still in **less** (i.e. don't press [q] to quit), type a forward slash **[/]** followed by the word to search (to search the word thermal try

/thermal

As you can see, **less** finds and highlights the keyword. Type **[n]** to search for the next occurrence of the word.

grep (don't ask why it is called grep)

grep is one of many standard Linux utilities. It searches files for specified words or patterns. First clear the screen, then type

```
$ grep thermal syslog.txt
```

As you can see, **grep** has printed out each line containing the word thermal.

Try typing

```
$ grep Thermal syslog.txt
```

The **grep** command is case sensitive; it distinguishes between thermal and Thermal.

To ignore upper/lower case distinctions, use the **-i** option, i.e. type

```
$ grep -i thermal syslog.txt
```

To search for a phrase or pattern, you must enclose it in single quotes (the apostrophe symbol). For example to search for 'Unable to find a sensor', type

```
$ grep -i 'Unable to find a sensor' syslog.txt
```

Some of the other options of grep are:

- v display those lines that do NOT match
- n precede each matching line with the line number
- c print only the total count of matched lines

Try some of them and see the different results. Don't forget, you can use more than one option at a time, for example, the number of lines without the words thermal is

```
$ grep -ivc thermal syslog.txt
```

wc (word count)

A handy little utility is the **wc** command, short for word count. To do a word count on **syslog.txt**, type

```
$ wc -w syslog.txt
```

To find out how many lines the file has, type

```
$ wc -l syslog.txt
```

Summary

command	meaning
cp <i>file1 file2</i>	copy file1 and call it file2
mv <i>file1 file2</i>	move or rename file1 to file2
rm <i>file</i>	remove a file
rmdir <i>directory</i>	remove a directory
cat <i>file</i>	display a file
more <i>file</i>	display a file a page at a time
head <i>file</i>	display the first few lines of a file
tail <i>file</i>	display the last few lines of a file
grep <i>'keyword' file</i>	search a file for keywords
wc <i>file</i>	count number of lines/words/characters in file

3.1 Redirection

Most processes initiated by Linux commands write to the standard output (that is, they write to the terminal screen), and many take their input from the standard input (that is, they read it from the keyboard). There is also the standard error, where processes write their error messages, by default, to the terminal screen.

We have already seen one use of the **cat** command to write the contents of a file to the screen.

Now type **cat** without specifying a file to read

```
$ cat
```

Then type a few words on the keyboard and press the **[Return]** key.

Finally hold the **[Ctrl]** key down and press **[d]** (written as ^D for short) to end the input.

What has happened?

If you run the **cat** command without specifying a file to read, it reads the standard input (the keyboard), and on receiving the end of file' (^D), copies it to the standard output (the screen).

In Linux, we can redirect both the input and the output of commands.

3.2 Redirecting the Output

We use the `>` symbol to redirect the output of a command. For example, to create a file called **list1** containing a list of fruit, type

```
$ cat > list1
```

Then type in the names of some fruit. Press **[Return]** after each one and type Control D to stop.

```
pear  
banana  
apple  
^D
```

What happens is the **cat** command reads the standard input (the keyboard) and the `>` redirects the output, which normally goes to the screen, into a file called **list1**

To read the contents of the file, type

```
$ cat list1
```

Exercise 3a

Using the above method, create another file called **list2** containing the following fruit: orange, plum, mango, grapefruit. Read the contents of **list2**

The form `>>` appends standard output to a file. So to add more items to the file **list1**, type

```
$ cat >> list1
```

Then type in the names of more fruit and type Control D to stop.

```
peach  
grape  
orange  
^D
```

To read the contents of the file, type

```
$ cat list1
```

You should now have two files. One contains six fruit, the other contains four fruit. We will now use the **cat** command to join (concatenate)

list1

and **list2** into a new file called **biglist**. Type

```
$ cat list1 list2 > biglist
```

What this is doing is reading the contents of **list1** and **list2** in turn, then outputting the text to the file **biglist**

To read the contents of the new file, type

```
$ cat biglist
```

3.3 Redirecting the Input

We use the < symbol to redirect the input of a command.

The command **sort** alphabetically or numerically sorts a list. Type

```
$ sort
```

Then type in the names of some vegetables. Press **[Return]** after each one.

```
carrot
beetroot
artichoke
^D (control d to stop)
```

The output will be

```
artichoke
beetroot
carrot
```

Using < you can redirect the input to come from a file rather than the keyboard. For example, to sort the list of fruit, type

```
$ sort < biglist
```

and the sorted list will be output to the screen.

To output the sorted list to a file, type,

```
$ sort < biglist > slist
```

Use **cat** to read the contents of the file **slist**

3.4 Pipes

For example, typing

```
$ cat biglist | sort
```

will give the same result as above, but quicker and cleaner.

To find out

```
$ cat biglist | wc -l
```

Summary

command	meaning
<code>command > file</code>	redirect standard output to a file
<code>command >> file</code>	append standard output to a file
<code>command < file</code>	redirect standard input from a file
<code>command1 command2</code>	pipe the output of command1 to the input of command2
<code>cat file1 file2 > file0</code>	concatenate file1 and file2 to file0
<code>sort</code>	sort data
<code>who</code>	list users currently logged in

4.1 Wildcards

The characters * and ?

The character * is called a wildcard, and will match against none or more character(s) in a file (or directory) name. For example, in your `icfoss` directory, type

```
$ ls list*
```

This will list all files in the current directory starting with `list....`

Try typing

```
$ ls *list
```

This will list all files in the current directory ending with.....`list`

The character ? will match exactly one character.

So `ls ?ouse` will match files like `house` and `mouse`, but not `grouse`.

Try

typing

```
$ ls ?list
```

4.2 Filename conventions

We should note here that a directory is merely a special type of file. So the rules and conventions for naming files apply also to directories.

In naming files, characters with special meanings such as `/ * & $ ,` should be avoided. Also, avoid using spaces within names. The safest way to name a file is to use only alphanumeric characters, that is, letters and numbers, together with `_` (underscore) and `.` (dot).

File names conventionally start with a lower-case letter, and may end with a dot followed by a group of letters indicating the contents of the

file. For example, all files consisting of C code may be named with the ending .c, for example, prog1.c . Then in order to list all files containing C code in your home directory, you need only type **ls *.c** in that directory.

Beware: some applications give the same name to all the output files they generate.

For example, some compilers, unless given the appropriate option, produce compiled files named **a.out**. Should you forget to use that option, you are advised to rename the compiled file immediately, otherwise the next such file will overwrite it and it will be lost.

4.3 Getting Help

On-line Manuals

There are on-line manuals which gives information about most commands. The manual pages tell you which options a particular command can take, and how each option modifies the behaviour of the command. Type **man** command to read the manual page for a particular command.

For example, to find out more about the **wc** (word count) command, type

```
$ man wc
```

Alternatively

```
$ whatis wc
```

gives a one-line description of the command, but omits any information about options etc.

Summary

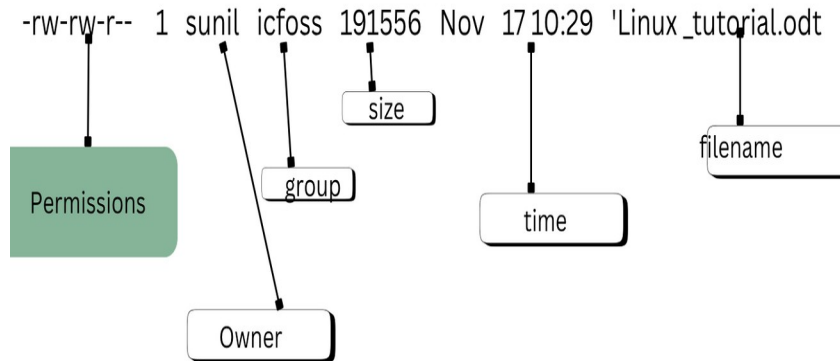
command	meaning
*	match any number of characters
?	match one character
man command	read the online manual page for a command
whatis command	brief description of a command
apropos keyword	match commands with keyword in their man pages

In your icfoss directory, type

```
$ ls -l
```

The `l` is for *long listing*.

You will see that you now get lots of details about the contents of your directory, similar to the example below. (Make sure you understand this)



Each file (and directory) has associated access rights for the user(vu2swx), the group(icfoss), and others, which may be found by typing `ls -l`:

```
-rwxrw-r-- 1 sunil icfoss 2450 Sept29 11:52 file1
```

In the left-hand column is a 10 symbol string consisting of the symbols `d`, `r`, `w`, `x`, `-`, and, occasionally, `s` or `S`. If `d` is present, it will be at the left hand end of the string, and indicates a directory: otherwise `-` will be the starting symbol of the string.

The 9 remaining symbols indicate the permissions, or access rights, and are taken as three groups of 3.

- The left group of 3 gives the file permissions for the user that owns the file (or directory) (ee51ab in the above example);
- the middle group gives the permissions for the group of people to whom the file (or directory) belongs (beng95 in the above example);
- the rightmost group gives the permissions for all others.

The symbols `r`, `w`, etc., have slightly different meanings depending on whether they refer to a simple file or to a directory.

Access rights on files.

- `r` (or `-`), indicates read permission (or otherwise), that is, the presence or absence of permission to read and copy the file
- `w` (or `-`), indicates write permission (or otherwise), that is, the permission (or otherwise) to change a file
- `x` (or `-`), indicates execution permission (or otherwise), that is, the permission to execute a file, where appropriate

Access rights on directories.

- r allows users to list files in the directory;
- w means that users may delete files from the directory or move files into it;
- x means the right to access files in the directory. This implies that you may read files in the directory provided you have read permission on the individual files.

So, in order to read a file, you must have execute permission on the directory containing that file, and hence on any directory containing that directory as a subdirectory, and so on, up the tree.

Some examples

<code>-rwxrwxrwx</code>	a file that everyone can read, write and execute (and delete).
<code>-rw-----</code>	a file that only the owner can read and write - no-one else can read or write and no-one has execution rights (e.g. your mailbox file).

5.2 Changing access rights

chmod (changing a file mode)

Only the owner of a file can use **chmod** to change the permissions of a file. The options of **chmod** are as follows

Symbol	Meaning
u	user
g	group
o	other
a	all
r	read
w	write (and delete)
x	execute (and access directory)
+	add permission
-	take away permission

For example, to remove read write and execute permissions on the file **biglist** for the group and others, type

```
$ chmod go-rwx biglist
```

This will leave the other permissions unaffected.

To give read and write permissions on the file **biglist** to all,

```
$ chmod a+rw biglist
```

Exercise 5a

Try changing access permissions on the file **syslog.txt** and on the directory **backups**

Use `ls -l` to check that the permissions have changed.

5.3 Processes and Jobs

A process is an executing program identified by a unique PID (process identifier). To see information about your processes, with their associated PID and status, type

```
$ ps
```

A process may be in the foreground, in the background, or be suspended. In general the shell does not return the UNIX prompt until the current process has finished executing.

Some processes take a long time to run and hold up the terminal. Backgrounding a long process has the effect that the UNIX prompt is returned immediately, and other tasks can be carried out while the original process continues executing.

Running background processes

To background a process, type an **&** at the end of the command line. For example, the command **sleep** waits a given number of seconds before continuing. Type

```
$ sleep 10
```

This will wait 10 seconds before returning the command prompt **\$**. Until the command prompt is returned, you can do nothing except wait.

To run **sleep** in the background, type

```
$ sleep 10 &
```

```
[1] 6259
```

The **&** runs the job in the background and returns the prompt straight away, allowing you to run other programs while waiting for that one to finish.

The first line in the above example is typed in by the user; the next line, indicating job number and PID, is returned by the machine. The user is notified of a job number (numbered from 1) enclosed in square brackets, together with a PID and is notified when a background process is finished. Backgrounding is useful for jobs which will take a long time to complete.

Backgrounding a current foreground process

At the prompt, type

```
$ sleep 100
```

You can suspend the process running in the foreground by holding down the **[control]** key and typing **[z]** (written as **^Z**) Then to put it in the background, type

```
$ bg
```

Note: do not background programs that require user interaction e.g. vi

5.4 Listing suspended and background processes

When a process is running, backgrounded or suspended, it will be entered onto a list along with a job number. To examine this list, type

```
$ jobs
```

An example of a job list could be

```
[1] Suspended sleep 100
[2] Running netscape
[3] Running nedit
```

To restart (foreground) a suspended processes, type

```
$ fg $jobnumber
```

For example, to restart **sleep** 100, type

```
$ fg $1
```

Typing **fg** with no job number foregrounds the last suspended process.

5.5 Killing a process

kill (terminate or signal a process)

It is sometimes necessary to kill a process (for example, when an executing program is in an infinite loop)

To kill a job running in the foreground, type **^C** (control c). For example, run

```
$ sleep 100
^C
```

To kill a suspended or background process, type

```
$ kill $jobnumber
```

For example, run

```
$ sleep 100 &
$ jobs
```

If it is job number 4, type

```
$ kill $4
```


To check whether this has worked, examine the job list again to see if the process has been removed.

ps (process status)

Alternatively, processes can be killed by finding their process numbers (PIDs) and using **kill PID_number**

```
$ sleep 100 &
$ ps

PID TT S TIME COMMAND
20077 pts/5 S 0:05 sleep 100
21563 pts/5 T 0:00 netscape
21873 pts/5 S 0:25 nedit
```

To kill off the process **sleep 100**, type

```
$ kill 20077
```

and then type **ps** again to see if it has been removed from the list. If a process refuses to be killed, uses the **-9** option, i.e. type

```
$ kill -9 20077
```

Note: It is not possible to kill off other users' processes !!!

Summary

command	meaning
ls -lag	list access rights for all files
chmod [options] file	change access rights for named file
command &	run command in background
^C	kill the job running in the foreground
^Z	suspend the job running in the foreground
bg	background the suspended job
jobs	list current jobs
fg \$1	foreground job number 1
kill \$1	kill job number 1
ps	list current processes
kill 26152	kill process number 26152

Other useful Linux commands

quota

Each user on Henry2 has 1 GB of quota in their home directory. If you go over your quota, you will not be able to write any more files, giving an error of *No space left on device*.

To check your current quota and how much of it you have used, type

```
$ quota -s
```

df

The df command reports on the space left on the file system. For example, to find out how much space is left on the fileserver, type

```
$ df .
```

du

The du -h command outputs the number of MB used by each subdirectory. Useful if you have gone over quota and you want to find out which directory has the most files. In your home-directory, type

```
$ du -h
```

compress

This reduces the size of a file, thus freeing valuable disk space. For example, type

```
$ ls -l syslog.txt
```

and note the size of the file. Then to compress syslog.txt, type

```
$ compress syslog.txt
```

This will compress the file and place it in a file called syslog.txt.Z

To see the change in size, type ls -l again.

To uncompress the file, use the uncompress command.

```
$ uncompress syslog.txt.Z
```

gzip

This also compresses a file, and is more efficient than compress. For example, to zip syslog.txt, type

```
$ gzip syslog.txt
```

This will zip the file and place it in a file called syslog.txt.gz

To unzip the file, use the gunzip command.

```
$ gunzip syslog.txt.gz
```

file

file classifies the named files according to the type of data they contain, for example ascii (text), pictures, compressed data, etc.. To report on all files in your home directory, type

```
$ file *
```

history

The shell keeps an ordered list of all the commands that you have entered. Each command is given a number according to the order it was entered.

```
$ history (show command history list)
```

If you are using the bash shell, you can use the exclamation character (!) to recall commands easily.

```
$ !! (recall last command)
```

```
$ !-3 (recall third most recent command)
```

```
$ !5 (recall 5th command in list)
```

```
$ !grep (recall last command starting with grep)
```

You can increase the size of the history buffer by typing

6 Environment Variables

An example of an environment variable is the OSTYPE variable. The value of this is the current operating system you are using. Type

```
$ echo $OSTYPE
```

More examples of environment variables are

- USER (your login name)
- HOME (the path name of your home directory)
- HOST (the name of the computer you are using)
- ARCH (the architecture of the computers processor)
- DISPLAY (the name of the computer screen to display X windows)
- PRINTER (the default printer to send print jobs)
- PATH (the directories the shell should search to find a command)

Finding out the current values of these variables.

ENVIRONMENT variables are set using the **setenv** command, displayed using the **printenv** or **env** commands, and unset using the

unsetenv command.

To show all values of these variables, type

```
$ printenv | less
```

Shell Variables

An example of a shell variable is the HISTSIZE variable. The value of this is how many shell commands to save, allow the user to scroll back through all the commands they have previously entered. Type

```
$ echo $HISTSIZE
```

More examples of shell variables are

- HOME (the path name of your home directory)
- PATH (the directories the shell should search to find a command)

Finding out the current values of these variables.

SHELL variables are both set and displayed using the **set** command. They can be unset by using the **unset** command.

To show all values of these variables, type

```
$ set | less
```

login files

Each time you login to a Linux host, the system looks in your home directory for initialization files . .bashrc is a hidden file that is executed each time you login

Try viewing it.

```
$ cat .bashrc
```

Uploading the final output

When you are done open a terminal and type

```
HISTTIMEFORMAT="%d/%m/%y %T "
```

This will set the formatting for history command with date and time.

Then redirect your history to a file named `assignment1.txt`.

```
$ history >assignment1.txt
```

The final output will show all the commands you attempted. It will be somewhat like the following figure

(Even if you don't complete, or get errors in between, upload the history)

```
56 12/11/23 22:02:57 apt install dosbox
57 12/11/23 22:02:57 sudo apt install dosbox
58 12/11/23 22:02:57 clear
59 12/11/23 22:10:29 cd /var/log/
60 12/11/23 22:10:29 ls
61 12/11/23 22:10:37 cat syslog
62 12/11/23 22:10:43 cat boot.log
63 12/11/23 22:10:49 ls -l
64 12/11/23 22:15:01 history
65 12/11/23 22:15:30 history
66 12/11/23 22:15:34 man history
67 12/11/23 22:17:07 ls
68 12/11/23 22:17:08 cd
69 12/11/23 22:17:10 ls
70 12/11/23 22:18:27 cat /var/log/syslog
71 12/11/23 22:45:50 HISTTIMEFORMAT="%d/%m/%y %T "
72 12/11/23 22:45:54 history
```

Zip this file (`assignment1.txt`) to a `tar.gz` file using `gzip` and upload it to the google form provided. (find out from internet how to use `tar` and `gzip` to create zip a file in `tar.gz` format) .

Rename the final zip file to `a1_yourname_yourktuid.tar.gz`

(`a1` → assignment 1, `yourname`→yourfirstname, `yourktuid`→your ktu id

eg `a1_atul_meat12200.tar.gz`. We need the final upload in this format for automatic evaluation.)

Enjoy Learning. If you are short listed for interview, you need to know what you did here. You are free to refer to any resource. No further help will be provided from ICFOSS. Use your brain if you run into trouble with any steps listed above.